ML-Assignment 3

Gustaf Utterström

May 2026

1

If we use the definition of y_i in the instruction, we have the following expression for $\frac{\partial y_l}{\partial x_i}$:

$$\begin{aligned} y_l &= \sum_{j=1}^{m} W_{l,j} x_j + b_l \\ \frac{\partial y_l}{\partial x_i} &= W_{l,i} \end{aligned} \tag{1}$$

The chain rule gives:

$$\begin{aligned} \frac{\partial L}{\partial x_i} &= \sum_{l=1}^{n} \frac{\partial L}{\partial y_l} \frac{\partial y_l}{\partial x_i} \\ &= \sum_{l=1}^{n} W_{l,i} \frac{\partial L}{\partial y_l} \end{aligned} \tag{2}$$

This is the i-th product of the matrix multiplication $\mathbf{W}^T \frac{\partial L}{\partial \mathbf{y}}$. Therefore, we can write:

$$\frac{\partial L}{\partial \mathbf{x}} = \mathbf{W}^T \frac{\partial L}{\partial \mathbf{y}} \tag{3}$$

And the dimensions are right since: $\mathbf{W}^T \in R^{m*n}$ $\frac{\partial L}{\partial \mathbf{y}} \in R^{n*1}$ and hence $\mathbf{x} \in R^{m*1}$ as given. To find $\frac{\partial L}{\partial \mathbf{W}}$ start with differentiating equation 1 with respect to $\mathbf{W}_{i,j}$:

$$\begin{aligned} \frac{\partial y_l}{\partial W_{i,j}} &= x_j, \text{ when } i = l \\ \frac{\partial y_l}{\partial W_{i,j}} &= 0, \text{ when } i \neq l \end{aligned} \tag{4}$$

Therefore, we get:

$$\frac{\partial L}{\partial W_{i,j}} = \sum_{l=1}^{n} \frac{\partial L}{\partial y_l} \frac{\partial y_l}{W_{ij}} = \frac{\partial L}{\partial y_i} x_j \tag{5}$$

Since this is the i,j-th entry, we can rewrite this as:

$$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{x}^T \tag{6}$$

And the dimensions are: $\mathbf{y} \in R^{n*1}, \mathbf{x}^T \in R^{1*m}, \mathbf{W} \in R^{n*m}$ And for $\frac{\partial L}{\partial \mathbf{b}}$ we use equation 1 again:

$$\frac{\partial L}{\partial b_i} = \sum_{l=1}^{n} \frac{\partial L}{\partial y_l} \frac{\partial y_l}{b_i} = \sum_{l=1}^{n} \frac{\partial L}{\partial y_l} \delta_{l,i} = \frac{\partial L}{\partial y_i} \tag{7}$$

And we can rewrite this as:

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{y}} \tag{8}$$

With dimensions: $\mathbf{b} \in R^{n*1}, \mathbf{y} \in R^{n*1}$

2

To express $\mathbf{Y}$ we write:

$$\mathbf{Y} = [\mathbf{W}\mathbf{x}^1 + \mathbf{b}\ \mathbf{W}\mathbf{x}^2 + \mathbf{b} ... \mathbf{W}\mathbf{x}^N + \mathbf{b}] = \mathbf{W}\mathbf{X} + \mathbf{b}\mathbf{1}^T \tag{9}$$

Where $\mathbf{W} \in R^{n*m}$, $\mathbf{X} \in R^{m*N}$ and hence $\mathbf{WX} \in R^{n*N}$ and $\mathbf{b} \in R^{n*1}$ but will get the right shape if $\mathbf{b1^T} \in R^{n*N}$. For $\frac{\partial L}{\partial \mathbf{X}}$ we use equation 3:

$$\frac{\partial L}{\partial \mathbf{X}} = (\frac{\partial L}{\partial \mathbf{x}^1} ... \frac{\partial L}{\partial \mathbf{x}^N}) = (\mathbf{W}^T \frac{\partial L}{\partial \mathbf{y}^1} ... \mathbf{W}^T \frac{\partial L}{\partial \mathbf{y}^N}) = \mathbf{W}^T \frac{\partial L}{\partial \mathbf{Y}} \tag{10}$$

With dimensions: $\mathbf{W}^T \in R^{m*n}$ and $\frac{\partial L}{\partial \mathbf{Y}} \in R^{n*N}$ and hence the result will $\in R^{m*N}$, the same as $\mathbf{X}$. To express $\frac{\partial L}{\partial \mathbf{W}}$ we use equation 6 and the summation given in the instructions:

$$\frac{\partial L}{\partial \mathbf{W}} = \sum_{l=1}^{N} \frac{\partial L}{\partial \mathbf{y}^l} \left(\mathbf{x}^l\right)^T = [\frac{\partial L}{\partial \mathbf{y}^1} ... \frac{\partial L}{\partial \mathbf{y}^N}][\left(\mathbf{x}^1\right)^T ... \left(\mathbf{x}^N\right)^T]^T = \frac{\partial L}{\partial \mathbf{Y}} \mathbf{X}^T \tag{11}$$

With the right dimensions since $\frac{\partial L}{\partial \mathbf{Y}} \in R^{n*N}$ and $\mathbf{X}^T \in R^{N*m}$ and hence the result will be $\in R^{n*m}$, the same as $\mathbf{W}$. For $\frac{\partial L}{\partial \mathbf{b}}$ we use the equation given in 8 with the summation given in the instructions:

$$\frac{\partial L}{\partial \mathbf{b}} = \sum_{l=1}^{N} \frac{\partial L}{\partial \mathbf{y}^l} = \frac{\partial L}{\partial \mathbf{y}} \mathbf{1} \tag{12}$$

Where $\mathbf{1} \in R^{N*1}$ and $\frac{\partial L}{\partial \mathbf{y}} \in R^{n*N}$ and hence the result will be $\in R^{n*1}$ which is the same as for $\mathbf{b}$.

```
def fully_connected_forward(X, W, b):
    ...
    ones = np.ones((batch,1))
    Z = W@X + b@ones.T
    return Z
```

```
def fully_connected_backward(X, dldZ, W, b):
    ...
    ones=np.ones((batch,1))
    dldX = (W.T@dldZ).reshape(sz, order='F')
    dldW = dldZ@X.T
    dldb = dldZ@ones

    return dldX, dldW, dldb
```

3

The activation function of ReLu is defined by:

$$y_i = \begin{cases} x_i \text{ ; if } x_i > 0 \\ 0 \text{ ; if } x_i \leq 0 \end{cases} \tag{13}$$

Hence, the derivative:

$$\frac{\partial y_i}{\partial x_i} = \begin{cases} 1 \text{ ; if } x_i > 0 \\ 0 \text{ ; if } x_i \leq 0 \end{cases} \tag{14}$$

Since ReLu is applied element-wise, y_i depends only on x_i and not for $l \neq i$, this gives:

$$\frac{\partial L}{\partial x_i} = \sum_{l=1}^{n} \frac{\partial L}{\partial y_l} \frac{\partial y_l}{\partial x_i} = \frac{\partial L}{\partial y_i} \frac{\partial y_i}{\partial x_i} \tag{15}$$

Then, we can write the expression as:

$$\frac{\partial L}{\partial x_i} = \begin{cases} \frac{\partial L}{\partial y_i} \text{ ; if } x_i > 0 \\ 0 \text{ ; if } x_i \leq 0 \end{cases} \tag{16}$$

```
def relu_forward(X):
     Z = np.maximum(0,X)
    return Z

def relu_backward(X, dldZ):
    dldX = dldZ * (X > 0) # boolean mask
    return dldX
```

4

To find $\frac{\partial L}{\partial x_i}$ we use the definition of $L(\mathbf{x}, c)$ in the instructions (equation 14):

$$
\begin{aligned}
L(\mathbf{x}, c) &= -x_c + \log(\sum_{j=1}^{m} e^{x_j}) \\
\frac{\partial L}{\partial x_i} &= \begin{cases} -1 \text{ ; if } i = c \\ 0 \text{ ; o.w} \end{cases} + \frac{1}{\sum_{j=1}^{m} e_j^x} * e^{x_i} \\
&= \begin{cases} -1 \text{ ; if } i = c \\ 0 \text{ ; o.w} \end{cases} + y_i
\end{aligned}
\tag{17}
$$

Here the second equality follows from the chain rule and the third from the definition of y_i, the soft max function.

```
def softmaxloss_forward(x, labels):
    ...
    x_c = x[labels, np.arange(batch)]
    # labels is the row and np.arange(batch) gives the sample
    L = np.mean(-x_c + np.log(np.sum(np.exp(x), axis=0)))
    return L

def softmaxloss_backward(x, labels):
    ...
    y = np.exp(x)/np.sum(np.exp(x),axis=0)
    y[labels,np.arange(batch)]-=1
    #np.arange(batch) makes column of length #samples in batch
    dldx = y/batch
    dldx=dldx.reshape(sz,order='F')
    return dldx
```

5

```
def training(net, x, labels, x_val, labels_val,
             opts, make_plots=False):
    ...

    # equation 17
    # momentum = mu, dict, layer i,
    # shape 0 and parms s, weight or bias
    # opts['momentum'] is mu
    # grads[i][s] gives the gradient
    # in layer i for parameter s

    momentum[i][0][s] = (
        opts['momentum'] * momentum[i][0][s]
```

```
        + (1 - opts['momentum']) * grads[i][s]
    )

    # equation 20
    # net["layers"][i]["params"][s]
    # is the parameters - w_n or weights

    net["layers"][i]["params"][s] = (
        net["layers"][i]["params"][s]
        - opts['learning_rate'] * (
            momentum[i][0][s]
            + opts['weight_decay']
            * net["layers"][i]["params"][s]
        )
    )
```

6

To plot the learned kernels for the first convolution layer, the following code was used:

```
def kernels():

    kernels = net["layers"][1]["params"]["weights"]
    #print(kernels.shape) 5,5,1,16 - 5x5 filters,
    #1 input chanel (grey scale) and nbr of filters

    fig, axes = plt.subplots(2, 8, figsize=(14, 4))
    # Subplots where axes are the grid

    fig.suptitle(
        "Learned␣Kernels␣␣First␣Convolutional␣Layer␣(55,␣16␣filters)",
        fontsize=12, fontweight='bold'
    )

    for i, ax in enumerate(axes.flat):
    #flat makes 2d to 1d and we get 16 frames.
        k = kernels[:, :, 0, i]
        # pick one of the filters
        ax.imshow(k, cmap='gray')
        # ax is the subplot, we plot the learnd filters
        #, each number is a weight
        #the values/weights are represented
        #by image brightness, each kernel slides over the image
        ax.set_title(f'Filter␣{i+1}', fontsize=8)

    plt.tight_layout()
    plt.show()
```

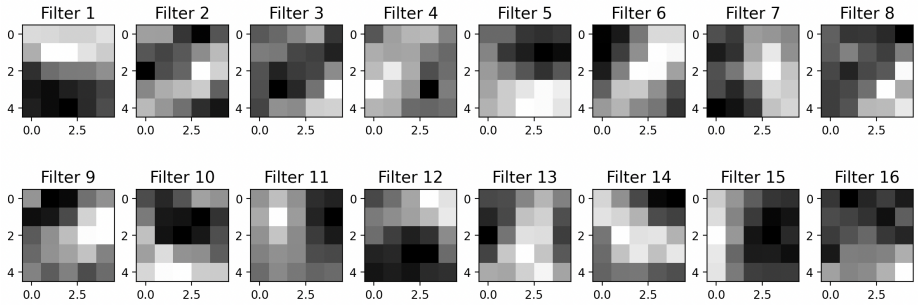


Figure 1: The 16 learned kernels/filter for the first convolution layer. The grayscale represents different weights/parameters. Brighter pixels means larger positive weight and darker corresponds to negative weights

In Figure 1 we expect to see filters that detect edges, lines, and curves since the model is trained on handwritten digits. The weights are adjusted to recognize digits from the Test set and generalize well. For example, filter 11 may detect and respond on vertical lines, to detect example digit 4. Filter 1 detects horizontal edges that may correspond to digit 5. Filters 6 and 14 may respond to diagonal structures in digit 7. The other filters capture patterns which may correspond to other digits, none of the filters looks noisy and do have a clear structure, this indicates a good CNN. To plot the misclassified images, the following code was used:

```
def missClass():
        missclass = []
        for i in range (len(y_test)):
            if y_test[i] != pred[i]:
                missclass.append(i)

        fig, axes = plt.subplots(4, 4, figsize=(8, 8))
        # Subplots where axes are the grid
        flat_ax = list(axes.flat)# len = 16

        for i in range (len(flat_ax)):
            ax = flat_ax[i]
            idx = missclass[i]
            img = x_test[:,:,0,idx]
            # takes every pixel in dim1 adn dim2 and
            #the channel, idx is image nbr
            ax.set_title(f"Pred:␣{int(pred[idx])},␣True:␣{int(y_test[idx])
                }")
            ax.imshow(img, cmap='gray')

        plt.suptitle("Misclassified␣MNIST␣samples")
        plt.tight_layout()
```

```
    plt.show()
missClass()
```

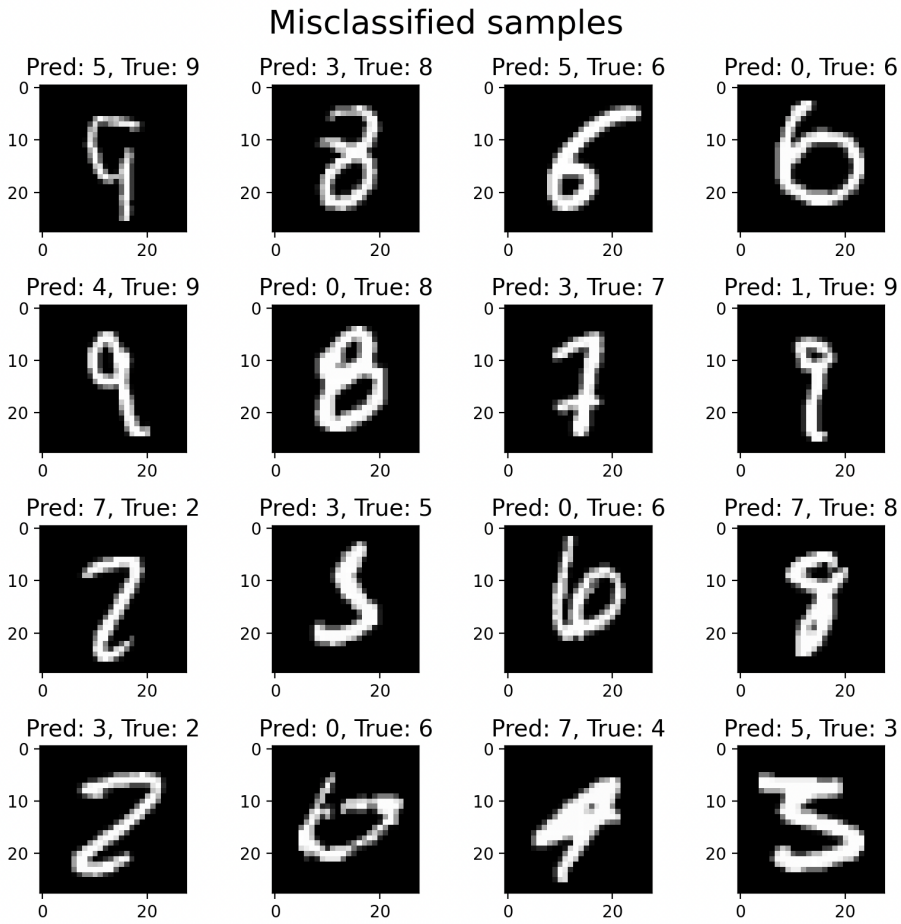


Figure 2: 16 misclassified images are presented in the figure above. The right label and the predicted label are stated.

In Figure 2 the limitations of the trained model are displayed. Although the weights and filters are well trained, variations in a specific class (digit) may be difficult to capture. In contrast, similarities between classes can be hard to distinguish. The reason for this may be poorly written digits, noisy images, or lack of weights (or training of weights). For example, digits 6 and 0 may have similar attributes, digits 2 and 7, and digits 3 or 5 may look similar. Some of the misclassifications in Figure 2 may also be misclassified by humans, this should be considered when evaluating the model. The code used to generate the confusion matrix and the metrics, precision, and recall are specified below:

```
def conf_matrix():
      con_m = confusion_matrix(y_test, pred)
      display = ConfusionMatrixDisplay(confusion_matrix=con_m)
      display.plot(cmap='Blues', values_format='d')
      plt.title("Confusion Matrix - MNIST CNN")
      plt.show()
  conf_matrix()

  ### Metrics###
  def metrics():
      precision = precision_score(y_true=y_test,y_pred=pred, average=
          None)
      recall = recall_score(y_true=y_test,y_pred=pred,average=None)
      for i in range(10):
          print(f"Digit {i}: Precision = {precision[i]:.3f}, Recall = {
              recall[i]:.3f}")
  metrics()
```

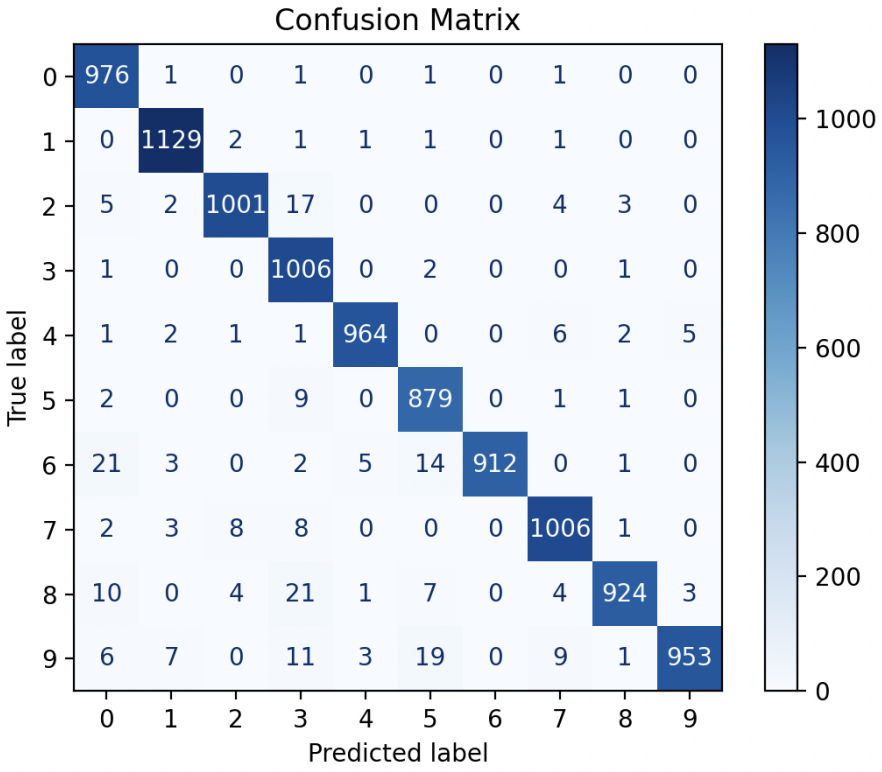


Figure 3: The confusion matrix for the predictions of the Test set

Digit	Precision	Recall
0	0.958	0.997
1	0.989	0.992
2	0.991	0.951
3	0.964	0.986
4	0.965	0.994
5	0.993	0.969
6	0.994	0.966
7	0.959	0.980
8	0.957	0.980
9	0.992	0.942

Table 1: Precision and Recall per digit for the Test Set

The confusion matrix has a strong diagonal, indicating that the model correctly classifies most of the samples. Notable is that the model misclassified digit 0 to be digit 6 for 21 instances, this is also commented on above and in Figure 2. The confusion matrix also suggests that digit 3 is misclassified for digit eight 21 times and digit two 17 times. This may be addressed to similar attributes, which is also verified in Figure 2. From Table 1, digits 0 and 8 have the lowest precision, meaning that the predictions of these digits are more likely to be confused with others. The digits with the lowest recall are 9 and 2, meaning that the model does not correctly identify some actual instances of these digits. Once again, distinguishing between classes can be even difficult for the human eye, and therefore some errors are expected. The number of parameters is:

Layer	number of parameters
Convolution 1	5*5*16+16=416
Convolution 2	5*5*16*16+16=6416
Fully-connected	10*784+10=7850
Total:	14682

The number of parameters is relatively small and the complexity of the model is low. Therefore, the model can be trained efficiently and the risk of overfitting decreases. Since most of the parameters are in the fully connected layer, the main learning capacity is centered in this layer.

7

In my first improved version, I constructed a deeper architecture with the aim to make a model that generalizes better and learns more image features than the baseline model. I did this by changing the image filters from 5x5 pixels to 3x3. This reduces the number of parameters and hence overfitting. Smaller filters also enable for a more complex feature detection. Since I reduced the size, and parameters, for each filter they were cheaper to train and therefore I increased the number of filters. By doing this, I enable a more specific and wider range of

pattern recognition for each filter, and hence the model becomes more generalized. The progression of filters was also split up because the feature complexity increases with depth. I also added an extra convolution layer to detect more complex structures and objects. The first layer mainly learns the edges and corners, the second pattern, and by adding an extra layer, I can increase the complexity in the model. I also delayed the pooling because i wanted to keep the spatial resolution as high as possible to preserve information in each image and improve the classification accuracy. In the improved model, I also added a fully connected layer and did the mapping between features to classes in two steps. Basically, this enables non-linear combinations of leaned features and a better model. I also increased the batch size because I wanted to reduce the noise in the gradient and have a more stable optimization. Since I increased the number of filters, I also increased the number of parameters, and to prevent the model from overfitting, I decreased the weight decay parameter. This regularization improves the generalization. Finally, since this network was deeper than my baseline, i decreased the learning rate to have stable optimization during the training. This model gave a accuracy on the Test set of 0.52.

When I did the training on the previous model, I reached an acceptable accuracy with no sign of convergence. Therefore, I used the same setup, just tuning the hyper parameters. Since the previews models training was stable, I increased the learning rate, to allow faster training. By doing this, I was able to increase the number of iterations. Since a deeper network needs more optimization time to train the parameters, i thought this would be an appropriate upgrade. With this model, the accuracy becomes 0.6. The plots and numbers computed for this models i achieved with the same code as for exercise 6. The kernels in the first convolution layer are:

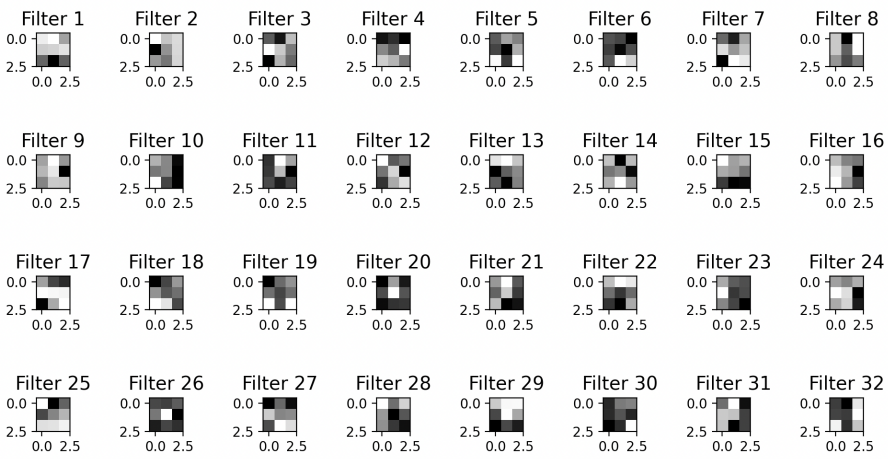


Figure 4: The 32 learned kernels/filter for the first convolution layer. The grayscale represents different weights/parameters. Brighter pixels means larger positive weight and darker corresponds to negative weights

As discussed in Figure 1 we expect Kernels that detect edges, lines, and curves in the first convolution layer. For example, Filter 10 and may detect vertical lines. Filters 1 and 15 detect the horizontal line. The other filters may capture patterns in other objectives represented in the images. Noticeable is that these are the filters after the first layer, more well trained filters will occur in the deeper layers.

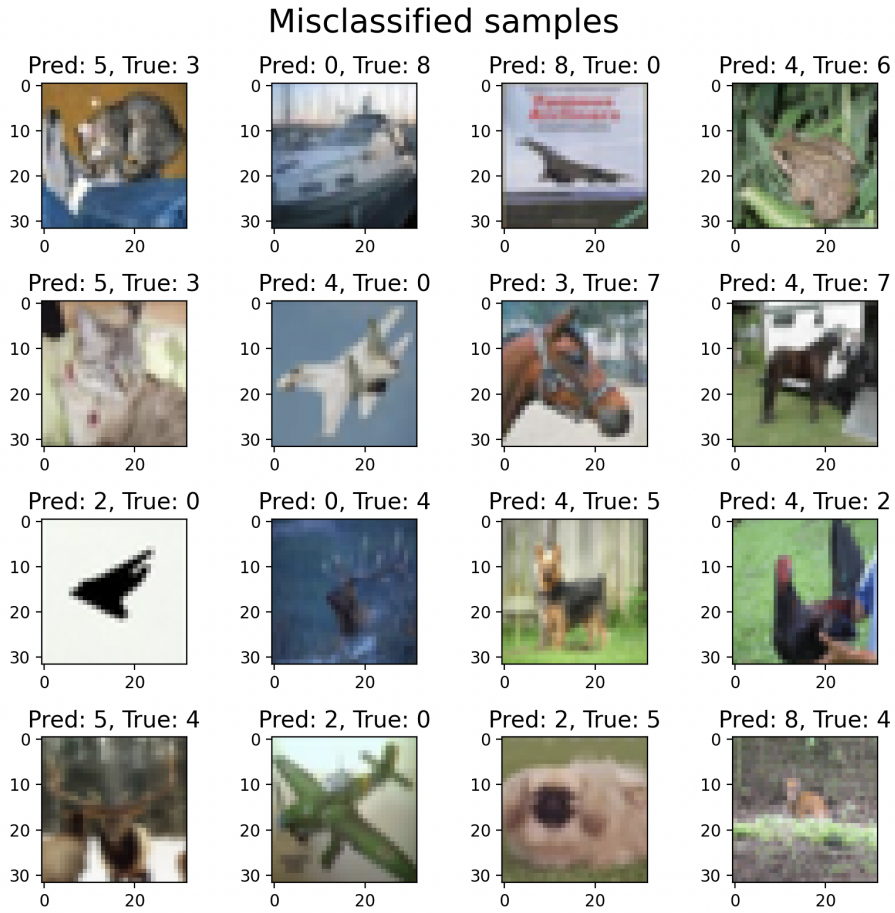


Figure 5: 16 misclassified images are presented in the figure above. The right label and the predicted label are stated.

Digit	Label
0:	Airplane
1:	Automobile
2:	Bird
3:	Cat
4:	Deer
5:	Dog
6:	Frog
7:	Horse
8:	Ship
9:	Truck

The misclassifications in Figure 5 maybe depends on similarities between classes, that are hard to distinguish. Noisy images may also be the reason for wrong classification. A cat can for instance have the same attribute as a dog. This kind of misclassifications is displayed in Figure 5, animals are mixed up with each other. The same holds for vehicles. The model has hard to distinguish birds and airplanes.

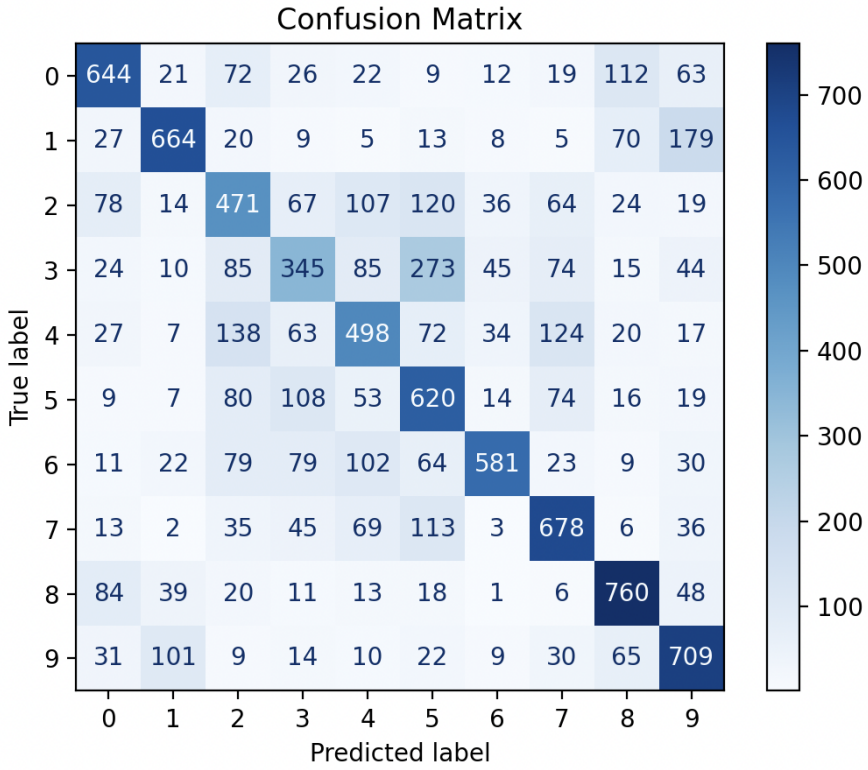


Figure 6: The confusion matrix for the predictions of the Test set

In Figure 5 the categorical misclassifications is clearly displayed. The reader can distinguish a square in the middle of the matrix, for row/column two to seven. This labels corresponds to animals. In the corners of the matrix a 2x2 square is also visible, the labels zero to two corresponds to vehicles as well as eight to nine. Therefore, we can expect this parts of the graph to have a higher misclassifications rate.

Digit	Precision	Recall
0	0.679	0.644
1	0.749	0.664
2	0.467	0.471
3	0.450	0.345
4	0.517	0.498
5	0.468	0.620
6	0.782	0.581
7	0.618	0.678
8	0.693	0.760
9	0.609	0.709

Table 2: Precision and Recall per label for the Test set

From Table 2 one can tell that label 8, ship, have the overall best performance. This implicates that the model is best to find patterns in ships and classify these. Label 2-4, or bird, cat and deer has the lowest precision and recall. This animals may have patterns that do not distinguish them from each other, or other objects. The variation in the classes may also be high, an therefore the completed training may not be sufficient.

Layer	Number of Parameters
Convolution 1	$3 * 3 * 3 * 32 + 32 = 896$
Convolution 2	$3 * 3 * 32 * 32 + 32 = 9248$
Convolution 3	$3 * 3 * 32 * 64 + 64 = 18496$
Fully-connected 1	$128 * (64 * 8 * 8) + 128 = 128 * 4096 + 128 = 524416$
Fully-connected 2	$10 * 128 + 10 = 1290$
Total	554346

The table above states that most of the parameters are concentrated in the first fully connected layer. This is because the $8*8*64$ feature maps are flattened to a 4096 dimensional vector. The first convolution layers have fewer parameters and are therefore efficient to train. Since the architecture has many convolution layers the model learns simpler features, while the deeper ones learns more object-specific patterns. This kind of CNN have a better classification accuracy compared to the simpler baseline model.